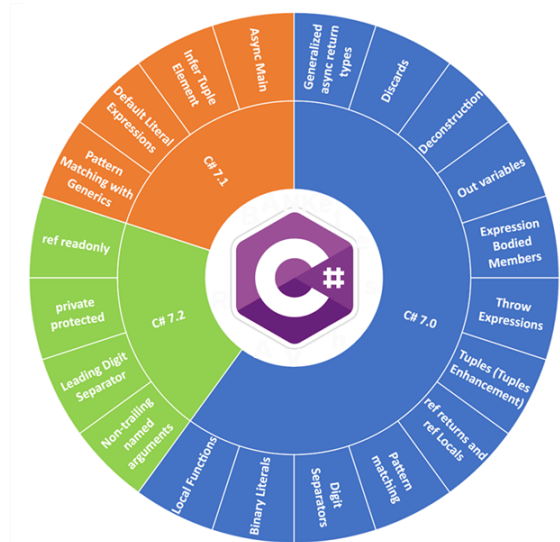


Sprachfeatures in C#

Modulübersicht

- C# 7.0
 - C# 7.1
 - C# 7.2
 - C# 7.3
- C# 8.0
- C# 9.0



Defaults

Target framework	version	C# language version default
.NET	5.x	C# 9.0
.NET Core	3.x	C# 8.0
.NET Core	2.x	C# 7.3
.NET Standard	2.1	C# 8.0
.NET Standard	2.0	C# 7.3
.NET Standard	1.x	C# 7.3
.NET Framework	all	C# 7.3

```
<PropertyGroup>  
  <LangVersion>7.2</LangVersion>  
</PropertyGroup>
```

.NET Framework vs. .NET Core

.NET Framework

- Windows exklusiv
- Closed Source
- Nicht modular
 - Ein großes Gesamtpaket mit allen Frameworks zusammen

<https://github.com/dotnet>

.NET Core

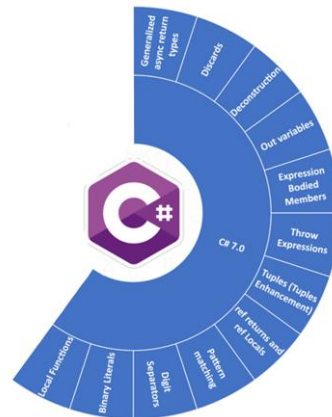
- Cross Platform
 - Windows, Linux, MacOS, ...
- Open Source
- Modular
 - ASP.NET Core, WPF, Xamarin/MAUI, UWP, ...
 - VS Installer
- Kommando Zeile

<https://dotnet.github.io/>

C# 7.0

C# 7.0 – Überblick

- out Variablen
- Pattern Matching
 - is – Operator mit Patterns
 - switch
- Tupel
- lokale Funktionen
- Separator für numerische Werte
- Return by Reference
- ValueTask<T>
- Mehr Expression Bodied Members
- Exceptions in Expressions werfen



out

```
private void IntCheck()
{
    string eingabe = "12345";
    int ausgabe; // <- deklaration erforderlich !
    if(Int32.TryParse(eingabe,out ausgabe))
    {
        Console.WriteLine(ausgabe);
    }
}
```

```
private void IntCheck()
{
    string eingabe = "12345";
    if(Int32.TryParse(eingabe,out var ausgabe))
    {
        Console.WriteLine(ausgabe);
    }
}
```

Discard von output:

`Int32.TryParse(eingabe,out _ )` // -> keine neue Variable, ergebnis rennt ins leere

Pattern Matching

```
private void SchreibSterneInDieKonsole(object o)
{
    if (o is null) // Constant pattern "null"
        return;
    if (o is int i) // Typenpattern
        Console.Write(new string('*', i));
}
```


Pattern Matching mit Switch

```
private void MusterabgleichMitSwitch(Grafik g)
{
    switch (g)
    {
        case Kreis k:
            Console.WriteLine($"Kreis mit dem Radius {k.Radius}");
            break;
        case Rechteck q when (q.Länge == q.Breite):
            Console.WriteLine($"Quadrat mit der Seitenlänge {q.Länge}");
            break;
        case Rechteck r:
            Console.WriteLine($"Rechteck mit der Länge {r.Länge} und Breite {r.Breite}");
            break;
        default:
            Console.WriteLine("Unbekannte Grafik");
            break;
    }
}
```

Tupel

```
class Person
{
    public string Vorname { get; set; }
    public string ZweiterVorname { get; set; }
    public string Nachname { get; set; }

    public (string, string, string) VollenNamenAusgabe()
    {
        return (Vorname, !string.IsNullOrEmpty(ZweiterVorname) ? ZweiterVorname : string.Empty, Nachname);
    }
}

class Program
{
    static void Main(string[] args)
    {
        Person p = new Person { Vorname = "Max", Nachname = "Mustermann" };
        var name = p.VollenNamenAusgabe();
        Console.WriteLine($"{name.Item1} {name.Item2} {name.Item3}");
    }
}
```

Technisch gesehen gibt's Tupel schon viel länger im .NET Framework, doch erst seit C# 7.0 kann man sie in der gezeigten Art und Weise nutzen.

Gründe für Tupel:

out ist, trotz der Verbesserung von vorher, immer noch meh // --- Herzeigen !

Man spart sich viel Overhead wenn man einfach nur eine reihe von Variablen ausgeben will

Annonym geht auch

Tupel

```
class Person
{
    public string Vorname { get; set; }
    public string ZweiterVorname { get; set; }
    public string Nachname { get; set; }

    public (string Vorname, string ZweiterVorname, string Nachname) VollenNamenAusgabe()
    {
        return (Vorname, !string.IsNullOrEmpty(ZweiterVorname) ? ZweiterVorname : string.Empty, Nachname);
    }
}

class Program
{
    static void Main(string[] args)
    {
        Person p = new Person { Vorname = "Max", Nachname = "Mustermann" };
        var name = p.VollenNamenAusgabe();
        Console.WriteLine($"{name.Vorname} {name.ZweiterVorname} {name.Nachname}");
    }
}
```

Alternative Schreibweise

Tupel Dekonstruktion

```
public void Main()
{
    Person p = new Person { Vorname = "Max", Nachname = "Mustermann" };

    (string vn, string zv, string nn) = p.VollenNamenAusgeben();    // Variante 1

    (var vn2, var zv2, var nn2) = p.VollenNamenAusgeben();          // Variante 2

    var (vn3, zv3, nn3) = p.VollenNamenAusgeben();                  // Variante 3

    string vorname, zweiterVorname, nachname;
    (vorname, zweiterVorname, nachname) = p.VollenNamenAusgeben();  // Variante 4

    Console.WriteLine($"{vorname} {zweiterVorname} {nachname}");
}
```

Dekonstruktion von Typen

```
class Kunde
{
    public int ID { get; set; }
    public string Name { get; set; }
    public bool Stammkunde { get; set; }

    public void Deconstruct(out int ID, out string Name, out bool Stammkunde)
    {
        ID = this.ID;
        Name = this.Name;
        Stammkunde = this.Stammkunde;
    }
}

public void Main()
{
    Kunde k = new Kunde { ID = 12, Name = "Max Mustermann", Stammkunde = true };
    var (id, name, stammkunde) = k;
    Console.WriteLine($"{id}: {name} ist Stammkunde:{stammkunde}");
}
```

Lokale Funktionen

```
public void WertVerdoppeln()  
{  
    int wert = 123;  
    void LokaleFunktion()  
    {  
        wert *= 2;  
    }  
    LokaleFunktion();  
    Console.WriteLine(wert);  
}
```

```
public int Fibonacci(int x)  
{  
    return Fib(x - 1).AktuellerWert;  
    (int AktuellerWert, int VorherigerWert) Fib(int i)  
    {  
        if (i == 0) return (1, 0);  
        var (p, pp) = Fib(i - 1);  
        return (pp + p, p);  
    }  
}
```

Hilfsmethoden, die nur für eine Methode bestimmt sind, werden in der betroffenen Methode „eingeschlossen“

Literale

```
public void Literale()
{
    int hex = 0x00FF00;
    byte bin = 0b00001111;
    byte bin2 = 0b0000_1111;
    int hundertdreiundzwanzig = 1____2____3;
}
```

Rückgabe per Referenz

```
public ref int Zahlensuche(int gesuchteZahl, int[] zahlen)
{
    for (int i = 0; i < zahlen.Length; i++)
    {
        if (zahlen[i] == gesuchteZahl)
            return ref zahlen[i];
    }
    throw new IndexOutOfRangeException();
}

public void Main()
{
    int[] zahlen = { 5, 7, 432, 567, -98, 3, 2 };
    ref int position = ref Zahlensuche(3, zahlen); // Alias für Index 5
    position = 100_000_000;
    Console.WriteLine(zahlen[5]);
}
```


ValueTask<T>

- C# 6.0 und darunter erlaubt nur void, Task oder Task<T> für asynchrone Methoden
- ValueTask<T> ist eine alternative für die Referenztypen Task und Task<T> und kann in einigen Situationen die Performance verbessern

Schleifen mit vielen awaits, es wird nicht immer ne memoryallocation gemacht usw ...

Mehr Expression Bodied Members

```
class Person
{
    public Person(string Name) => this.Name = Name;
    ~Person() => Name = null;

    private string name;
    public string Name
    {
        get => name;
        set => name = value;
    }
}
```

Exceptions in Expressions werfen

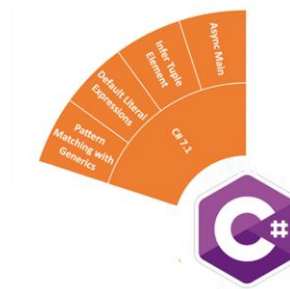
```
class Person
{
    public Person(string Name) => this.Name = Name ?? throw new ArgumentException();
    ~Person() => Name = null;

    public string Name { get; set; }
    public string GetVorname()
    {
        string[] geteilterName = Name.Split(' ');
        return (geteilterName.Length > 0) ? geteilterName[0] : throw new InvalidOperationException();
    }
    public string GetNachname() => throw new NotImplementedException();
}
```

C# 7.1

C# 7.1 – Überblick

- async Main
- Abgeleitete Tupелеlementnamen
- default Literale
- Pattern Matching mit Generics



Aktivieren geht nur, wenn man in den Optionen von „Latest Mayor Version“ auf die explizite C# - Version umstellt

async Main

```
class Program
{
    static async Task Main()
    {
        await Task.Delay(2000);

        Console.WriteLine("Hallo Welt");
    }
}
```

Abgeleitete Tupelelementnamen

```
static void Main()
{
    int zähler = 5;
    string label = "Farben in diesem Bild";
    var tupel = (zähler: zähler, label: label); // C# 7.0

    var tupel = (zähler, label);                // C# 7.1
}
```

BreakingChange mit einer Feature aus 7.0 !

default Literale

```
static int Main()
{
    Func<string, bool> whereFunktion = default(Func<string, bool>); // alt

    Func<string, bool> whereFunktion = default;                       // neu
    Func<int> defaultValue = () => default;
    int i = default;
    int[] zahlen = { default, 1, 2, 3 };

    if (i is default)
        Console.WriteLine("i hat den Standardwert");

    return default;
}

private void OptionalerParameter(int i = default)
{
}
```


Pattern Matching mit Generics

```
static void Print<T>(T input)
{
    switch (input)
    {
        case int i:
            Console.WriteLine($"Integer: {i}");
            break;
        case string s:
            Console.WriteLine($"String: {s}");
            break;
        default:
            Console.WriteLine("Unbekannter Typ");
            break;
    }
}
```

C# 7.2

C# 7.2 – Überblick

- readonly ref
- private protected
- Verbesserung beim Literal-Separator
- Verbesserung bei benannten Argumenten



readonly ref

- ref: Variable wird als Referenz übergeben
- out: Variable wird als Referenz übergeben und muss einen neuen Wert zugewiesen bekommen
- in: Variable wird als Referenz übergeben, ist aber readonly !

```
public static void InParameter(in int x)
{
    int y = x++;
}
```

(parameter) in int x

Cannot assign to variable 'in int' because it is a readonly variable

private protected

```
class Mitarbeiter
{
    public          int ID { get; set; }
    protected      string Name { get; set; }
    internal        DateTime Geburtsdatum { get; set; }
    private         decimal Gehalt { get; set; }
    protected internal int AbteilungsID { get; set; }

    private protected string Projektname { get; set; }
}
```

Quasi „private protected internal“ -> protected, aber nur für die eigene assembly !

Verbesserung beim Literal-Seperator

```
public void Literale()  
{  
    int hex = 0x00FF00;  
    byte bin = 0b00001111;  
    byte bin2 = 0b_0000_1111;  
    int hundertdreiundzwanzig = 1____2____3;  
}
```

```
public void Literale_CSharp7_2()  
{  
    int hex = 0x_00FF00;  
    byte bin = 0b__0000__1111;  
    // ...  
}
```

Verbesserung bei benannten Argumenten

```
public static long Summe(int zahl1, int zahl2 = default, int zahl3 = default)
{
    return zahl1 + zahl2 + zahl3;
}

public static void Main()
{
    int z1 = 5, z2 = 10, z3 = 15;

    long ergebnis = Summe(z1, z2, z3);           // geht
    long ergebnis2 = Summe(z1, zahl2: z2, zahl3: z3); // geht
    long ergebnis3 = Summe(z1, zahl2: z2, z3);    // geht seit C# 7.2
}
```

C# 7.3

C# 7.3 – Überblick:

- Neuzuweisung von lokalen ref – Variablen
- Arrayinitialisierer für stackalloc – Arrays
- Neue Constraints für generische Klassen
- Tupel unterstützen nun == und !=
- Attribute für automatisch generierte Felder

Neuzuweisung von lokalen ref - Variablen

```
SehrGroßesStruct sgs = new SehrGroßesStruct();  
SehrGroßesStruct sgs2 = new SehrGroßesStruct();  
  
ref SehrGroßesStruct referenz = ref sgs;  
referenz = ref sgs2;
```

Arrayinitialisierer für stackalloc – Arrays

```
unsafe
{
    int* pointerAufArray = stackalloc int[5] { 10, 20, 30, 40, 50 };
    int* pointerAufArray2 = stackalloc int[] { 10, 20, 30, 40, 50 };
    Span<int> beliebigeMenge = stackalloc [] { 10, 20, 30, 40, 50 };
}
```

Unsafe code ⓘ

☒ Allow code that uses the 'unsafe' keyword to compile.

<https://msdn.microsoft.com/de-de/magazine/mt814808.aspx>

System.Span<T> ist ein neuer Werttyp im Herzen von .NET. Er ermöglicht die Darstellung von zusammenhängenden Bereichen beliebigen Speichers, unabhängig davon, ob dieser Speicher einem verwalteten Objekt zugeordnet ist, ob er durch nativen Code über Interop bereitgestellt wird oder sich auf dem Stapel befindet. Und das alles bei gleichzeitigem sicheren Zugriff mit Leistungsmerkmalen wie bei Arrays.

Neue Constraints für generische Klassen

```
class InterfaceConstraint<T>    where T : IDisposable { }
class KlassenConstraint<T>      where T : Beispielklasse { }
class WertetypConstraint<T>     where T : struct { }
class ReferenztypConstraint<T>  where T : class { }
class KonstruktorConstraint<T>  where T : new() { }

class EnumConstraint<T>         where T : Enum { };
class DelegateConstraint<T>     where T : Delegate { };
class UnmanagedConstraint<T>   where T : unmanaged { };
```

Tupel unterstützen nun == und !=

```
var tupel1 = (12, "Hallo Welt");  
var tupel2 = (12, "Hallo Welt");  
  
if(tupel1 == tupel2)  
    Console.WriteLine("Gleich");
```

Attribute für automatisch generierte Felder

```
[Serializable]
class MeineKlasse
{
    [field: NonSerialized]
    public string Vorname { get; set; }
}
```

C# 8.0

C# 8.0 – Überblick:

- Readonly Member
- Mehr Expressions für gängige Patterns -> Switch/Property/Position/Tupel - Pattern
- Using Deklaration
- Statische lokale Funktionen
- Asynchrone Streams
- Indizes und Ranges
- Null-Coalescing Zuweisung
- Verbatim-Strings und String-Interpolation
- Referenztypen ohne null
- Neue Features für Interfaces

ReadOnly Member

```
public struct Punkt
{
    3 references
    public int X { get; set; }
    3 references
    public int Y { get; set; }
    0 references
    public readonly double Entfernung => Math.Sqrt(X * X + Y * Y); // Verändert keine Werte
    0 references
    public readonly void Berechne(int xOffset, int yOffset) // Verändert Werte
    {
        X += xOffset;
        Y += yOffset;
    }
}
```

`int Punkt.Y { get; set; }`
Cannot assign to "Y" because it is read-only

Compiler meckert wenn in einem readonly-Member Werte verändert werden.

Unterschied zu C#7 : Vorher konnte man nur ein „readonly struct“ erstellen, jetzt kann man das auch bei einzelnen Teilen machen

Mehr Expressions für gängige Patterns

- Switch - Pattern

```
public string HeutigerTag()
{
    switch (DateTime.Now.DayOfWeek)
    {
        case DayOfWeek.Monday:
            return "Montag";
        case DayOfWeek.Tuesday:
            return "Dienstag";
        case DayOfWeek.Wednesday:
            return "Mittwoch";
        case DayOfWeek.Thursday:
            return "Donnerstag";
        case DayOfWeek.Friday:
            return "Freitag";
        default:
            return "Wochenende";
    }
}
```

```
public string HeutigerTag()
{
    return DateTime.Now.DayOfWeek switch
    {
        DayOfWeek.Monday => "Montag",
        DayOfWeek.Tuesday => "Dienstag",
        DayOfWeek.Wednesday => "Mittwoch",
        DayOfWeek.Thursday => "Donnerstag",
        DayOfWeek.Friday => "Freitag",
        _ => "Wochenende"
    };
}
```

Mehr Expressions für gängige Patterns

- Property - Pattern

```
public decimal Bonuszahlung(Person mitarbeiter, decimal grundbetrag)
{
    return mitarbeiter switch
    {
        { Alter: 20 } => grundbetrag * 0.50m,
        { Alter: 21 } => grundbetrag * 0.60m,
        { Alter: 22 } => grundbetrag * 0.70m,
        { Alter: 23 } => grundbetrag * 0.80m,
        { Alter: 24 } => grundbetrag * 0.90m,
        { Alter: 25 } => grundbetrag,
        _ => grundbetrag * 2.0m // Alle anderen
    };
}
```

Mehr Expressions für gängige Patterns

- Position - Pattern

```
public decimal Bonuszahlung(Person mitarbeiter, decimal grundbetrag)
{
    return mitarbeiter switch
    {
        var (vn, nn, a) when vn == "Hannes" => grundbetrag * 5.00m,
        var (vn, nn, a) when a < 20 => grundbetrag * 1.25m,
        var (vn, nn, a) when a < 30 => grundbetrag * 1.50m,
        var (vn, nn, a) when a < 40 => grundbetrag * 2.00m,
        var (vn, nn, a) when a < 50 => grundbetrag * 2.50m,
        _ => grundbetrag * 3.00m
    };
}
```

Funktioniert mit allen Datentypen, die sich Dekonstruieren lassen (C#7er Feature)
In Klasse Person:

```
public void Deconstruct (out string vorname, out string nachname, out byte alter)
{
    vorname = Vorname;
    nachname = Nachname;
    alter = Alter;
}
```

Mehr Expressions für gängige Patterns

- Tupel - Pattern

```
public static string SchereSteinPapier(string element1, string element2)
=> (element1, element2) switch
{
    ("stein", "papier") => "Papier gewinnt",
    ("papier", "stein") => "Papier gewinnt",
    ("stein", "schere") => "Stein gewinnt",
    ("schere", "stein") => "Stein gewinnt",
    ("papier", "schere") => "Schere gewinnt",
    ("schere", "papier") => "Schere gewinnt",
    (_, _) => "Unentschieden"
};
```

Using Deklaration

```
public int SchreibeInEineDateiCSharp7X(string[] zeilen)
{
    int geschriebeneZeilen = 0;

    using (var datei = new StreamWriter("Demo.txt"))
    {
        foreach (string line in zeilen)
        {
            datei.WriteLine(line);
            geschriebeneZeilen++;
        }
    } // .Dispose()

    return geschriebeneZeilen;
}
```

```
public int SchreibeInEineDateiCSharp8(string[] zeilen)
{
    using var datei = new StreamWriter("Demo.txt");

    int geschriebeneZeilen = 0;
    foreach (string line in zeilen)
    {
        datei.WriteLine(line);
        geschriebeneZeilen++;
    }

    return geschriebeneZeilen;
} // .Dispose()
```

Man beachte, dass einmal „geschriebeneZeilen“ innerhalb und einmal außerhalb des Using-Blocks ist.

Statische lokale Funktionen

```
public int Berechnung()
{
    int y = 5;
    int x = 10;
    return Add(x, y);

    static int Add(int links, int rechts) => links + rechts;
}
```

Verhindern, dass man innerhalb der lokalen Funktionen auf Variablen des umschließenden Codeblocks zugreift.

Asynchrone Streams

```
public static async IEnumerable<int> GeneriereZahlen()
{
    for (int i = 0; i < 20; i++)
    {
        await Task.Delay(100);
        yield return i;
    }
}

public static async void GebeZahlenAus()
{
    await foreach (var zahl in GeneriereZahlen())
    {
        Console.WriteLine(zahl);
    }
}
```

Insbesondere für foreach relevant -> Ausführung der Schleife kann nun awaited werden

Indizes und Ranges

- Neue Syntax für das Arbeiten mit Sequenzen
 - System.Index: Index in einer Sequenz
 - ^ - Operator: Index relativ zum Ende der Sequenz
 - System.Range: Teilbereich einer Sequenz
 - .. - Operator: Beginn und Ende eines Teilbereiches

```
var wörter = new string[]
{
    "Franz",           // Index vom Anfang   Index vom Ende
                        // 0                      ^9
    "jagt",             // 1                      ^8
    "im",               // 2                      ^7
    "komplett",         // 3                      ^6
    "verwahrlosten",   // 4                      ^5
    "Taxi",             // 5                      ^4
    "quer",             // 6                      ^3
    "durch",           // 7                      ^2
    "Bayern"           // 8                      ^1
};                    // 9 (wörter.Length) ^0
```

Indizes und Ranges

- Neue Syntax für das Arbeiten mit Sequenzen
 - System.Index: Index in einer Sequenz
 - ^ - Operator: Index relativ zum Ende der Sequenz
 - System.Range: Teilbereich einer Sequenz
 - .. - Operator: Beginn und Ende eines Teilbereiches

```
// Franz jagt im komplett verwahrlosten Taxi quer durch Bayern

Console.WriteLine($"Das letzte Wort ist {wörter[^1]}");
var imKomplettVerwahrlostenTaxi = wörter[2..6]; // exkl wörter[6]
var durchBayern = wörter[^2..^0];             // exkl wörter[^0]
var alles = wörter[..];
var ersterTeil = wörter[..4]; // Franz jagt im komplett
var letzterTeil = wörter[6..]; // quer durch Bayern

Range phrase = 1..4;
var text = wörter[phrase]; // jagt im komplett
```

Null-Coalescing Zuweisung

- Zuweisungsoperation nur dann ausführen, wenn der linke Operator null ist

```
List<int> meineZahlen = null;
int? zahl = null;

meineZahlen ??= new List<int>(); // Wenn null, dann neue Liste erstellen
meineZahlen.Add(zahl ??= 17);    // Wenn null, dann 17 zuweisen
meineZahlen.Add(zahl ??= 20);    // Wenn null, dann 20 zuweisen

Console.WriteLine(string.Join(" ", meineZahlen));
// > 17 17
```

Verbatim-Strings und String-Interpolation

```
Console.WriteLine($"Diese Variante funktioniert");  
Console.WriteLine(@"Diese Variante funktioniert nun ebenfalls");
```

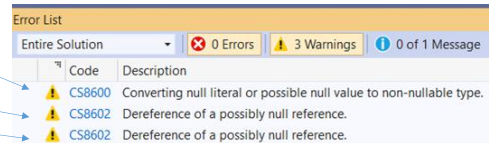
Referenztypen ohne null

```
static void Main(string[] args)
{
    #nullable enable
    string darfNotNullSein = null;
    string? darfNullSein = null;

    Console.WriteLine(darfNotNullSein[0]);
    Console.WriteLine(darfNullSein[0]);

    darfNotNullSein = "Demo";
    Console.WriteLine(darfNotNullSein);

    if(darfNullSein != null)
        Console.WriteLine(darfNullSein);
    #nullable disable
}
```



Error List	
Entire Solution	
0 Errors 3 Warnings 0 of 1 Message	
Code	Description
CS8600	Converting null literal or possible null value to non-nullable type.
CS8602	Dereference of a possibly null reference.
CS8602	Dereference of a possibly null reference.

<https://docs.microsoft.com/en-us/dotnet/csharp/nullable-references>

Anmerkungen:

Sobald ein Wert gesetzt wird oder ein Nullcheck ausgeführt wird, erkennt der Compiler, dass im weiteren Programmverlauf der Wert nicht mehr null sein kann.

Neue Features für Interfaces

- In Interfaces dürfen von nun an folgende Schlüsselwörter und Features genutzt werden:
- Zugriffsmodifizierer:
 - public, private, protected, internal, abstract, virtual
- Features
 - statische Felder und Methoden
 - statische Konstruktoren
 - partielle Interfaces
 - Code in Methoden (-> Standardimplementierung)

private und protected – Methoden setzen voraus, dass es eine Standardimplementierung gibt (siehe nächste Folie)

Standardimplementierung für Interfaces

```
interface IInterfaceMitDefaultMethode
{
    1 reference
    void MachEtwasInDerKlasse();
    0 references
    void MachEtwasImInterface()
    {
        Console.WriteLine("Code in einem Interface");
    }
}

class Demo : IInterfaceMitDefaultMethode
{
    // MachEtwasImInterface muss nicht implementiert werden !
    1 reference
    public void MachEtwasInDerKlasse()
    {
        throw new NotImplementedException();
    }
}
```

You can now add members to interfaces and provide an implementation for those members. This language feature enables API authors to add methods to an interface in later versions without breaking source or binary compatibility with existing implementations of that interface. Existing implementations *inherit* the default implementation. This feature also enables C# to interoperate with APIs that target Android or Swift, which support similar features. Default interface methods also enable scenarios similar to a "traits" language feature.

Standardimplementierung für Interfaces

```
interface IInterfaceMitDefaultMethode
{
    1 reference
    void MachEtwasInDerKlasse();
    0 references
    void MachEtwasImInterface()
    {
        Console.WriteLine("Code in einem Interface");
    }
}
```

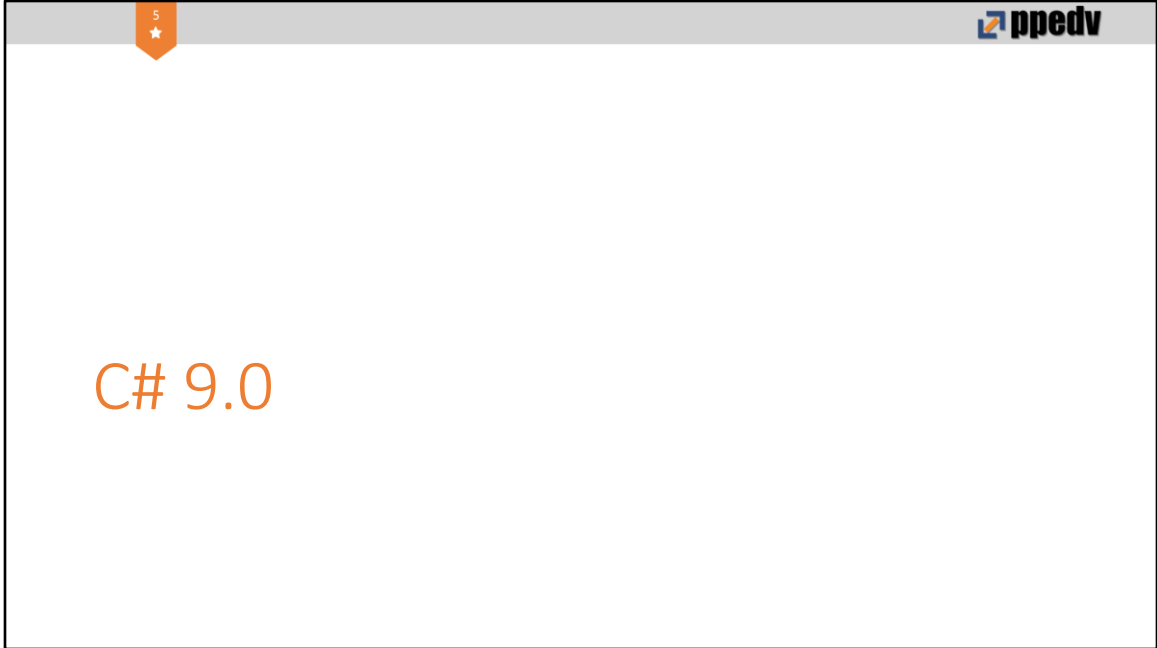
```
Demo d1 = new Demo();
d1.
```

- Equals
- GetHashCode
- GetType
- MachEtwasInDerKlasse
- ToString

```
IInterfaceMitDefaultMethode d1 = new Demo();
d1.
```

- Equals
- GetHashCode
- GetType
- MachEtwasImInterface
- MachEtwasInDerKlasse
- ToString

Anwendungsfall: Man hat ein Interface, das Xfach verwendet wird und man muss es um eine Methode erweitern -> Breaking-Change für alle, die diese dll nutzen



<https://docs.microsoft.com/en-us/dotnet/csharp/whats-new/csharp-9>

C# 9 – Überblick

- Top-Level Statements
- Target-typed New Expression
- Init only setters
- Patterns
 - Relational Patterns
 - Pattern Combinators
 - Switch Expression
- records

Top-Level Statements

- Namespace (optional)
 - class
 - static void Main(args)
- Niemand ruft die Main() auf
- Args sind einfach da
- Darf nur in einer Datei vorkommen

```
using System;

Console.WriteLine("Hello World!");

Console.WriteLine($"Programmargumente: {string.Join(", ", args)}");

Hallo();

void Hallo()
{
    Console.WriteLine("Hallo");
}

enum Versionen
{
    CSharp7,
    CSharp7_1,
    CSharp7_2,
    CSharp7_3,
    CSharp8,
    CSharp9,
    CSharp10,
    CSharp11
}
```

Target-typed New Expression

- Alternative zu var
- Lange Typen abkürzen

```
Person person = new Person(1, "Trainer");  
Person person2 = new(2, "Target-Typed new Trainer");  
var person3 = new Person(3, "var Trainer");  
  
Dictionary<int, Person> personen = new Dictionary<int, Person>();  
Dictionary<int, Person> personen = new();
```

Init only setters

- Wert kann nur einmal gesetzt werden
- private set
 - ctor
 - Kann geändert werden
- Verwendung in records

```
Person p = new Person(1, "Trainer");  
p.ID = 2; //Nicht erlaubt  
  
public class Person  
{  
    public int ID { get; init; } = 1; //Erlaubt  
  
    string Vorname { get; set; }  
  
    public Person(int ID, string Vorname)  
    {  
        this.ID = ID; //Erlaubt  
        this.Vorname = Vorname;  
    }  
}
```

Patterns

- If / else if / else if / else if / else
- Relational Expressions
 - >, <, >=, <=
- Pattern Combinators
 - is, and, or, not,

```
bool IstBuchstabe(char c) =>
    (c is >= 'a' and c is <= 'z') or
    (c is >= 'A' and c is <= 'Z');

if (wetter is not null)
{
    Console.WriteLine("Es gibt Wetter");
}
```

```
int temperatur = 4;

string wetter = temperatur switch
{
    < 0 => "Frost",
    > 0 and < 10 => "Kalt",
    > 10 and < 20 => "Frisch",
    > 20 and < 30 => "Warm",
    > 30 and < 35 => "Heiß",
    > 35 => "Hitze",
    _ => "Unbekannt"
}
```

Records

- Referenztyp (class)
- ToString()
- == prüft alle Werte
- GetHashCode
- Copy & Clone
- Vererbung nur Records
- Kann geöffnet werden

```
public class Person
{
    public int ID { get; init; }
    string Vorname { get; init; }

    public Person(int ID, string Vorname)
    {
        this.ID = ID;
        this.Vorname = Vorname;
    }
}

public record Person(int ID, string Vorname);

Person p = new Person(1, "Trainer");
```

